



Tecnológico de Monterrey

Reporte:

Proyecto 4: Generador de Código

Alumnos:

Gabriel Muñoz Luna - A01028774

Rodrigo Sosa Rojas - A01027913

Fecha de entrega:

Junio 4, 2026

Profesor:

Jorge Rodríguez Ruiz

Sección I: Código generado para MIPS

Nuestro generador de código convierte código de C- a código ensamblador MIPS, no hubo una razón específica para seleccionarlo más que teníamos mucha documentación y un poco de práctica gracias a las actividades hechas en clase junto al profesor.

Sección II: Manual de Usuario para el manejo del generador de código

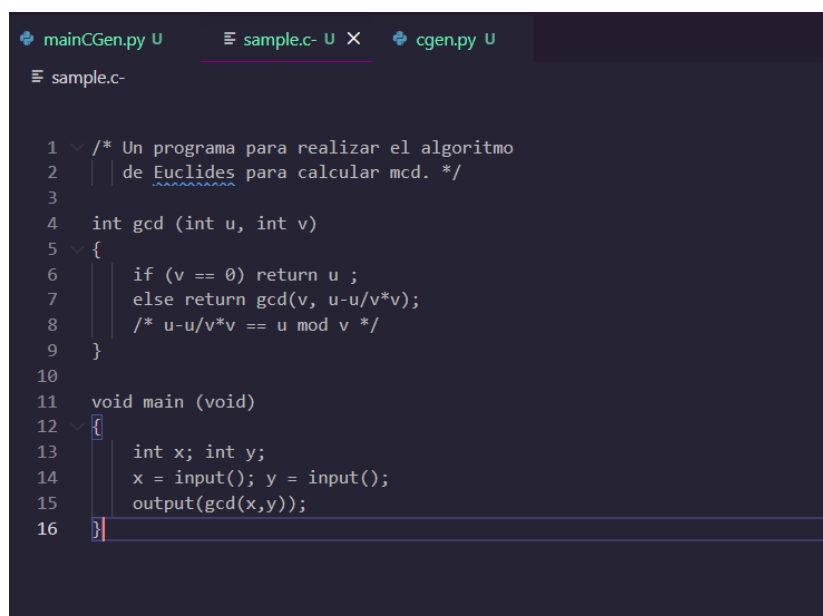
Paso 1:

Utilizamos la herramienta de MARS para correr nuestro código de MIPS, es necesario instalarlo desde la siguiente liga (Es necesario tener instalado Java para que MARS funcione):

[Instalador de MARS](#)

Paso 2:

Antes de correr el código, se necesita tener un archivo de C- que siga el nombre de “sample.c-”. Este código puede tener lo que sea de código, y debe insertarse en la carpeta donde todos los archivos .py coexisten, pues si no se hace de esta manera, el código no sabrá que archivo traducir.



```
mainCGen.py U  sample.c- U X  cgen.py U
sample.c-

1  /* Un programa para realizar el algoritmo
2  |   de Euclides para calcular mcd. */
3
4  int gcd (int u, int v)
5  {
6      if (v == 0) return u ;
7      else return gcd(v, u-u/v*v);
8      /* u-u/v*v == u mod v */
9  }
10
11 void main (void)
12 {
13     int x; int y;
14     x = input(); y = input();
15     output(gcd(x,y));
16 }
```

Al correr el código de Python, se necesita correr el mainCgen.py, donde automáticamente generará un archivo llamado “file.s” que será cargado directamente en el MARS.

mainCgen.py se encargará de crear la declaración de Tokens del Lexer, el AST del Parser, la tabla de símbolos del Analizador Semántico y el código traducido listo para ser utilizado.

No es necesario correr ningún otro código. Al finalizar, el archivo traducido tendrá la siguiente estructura:

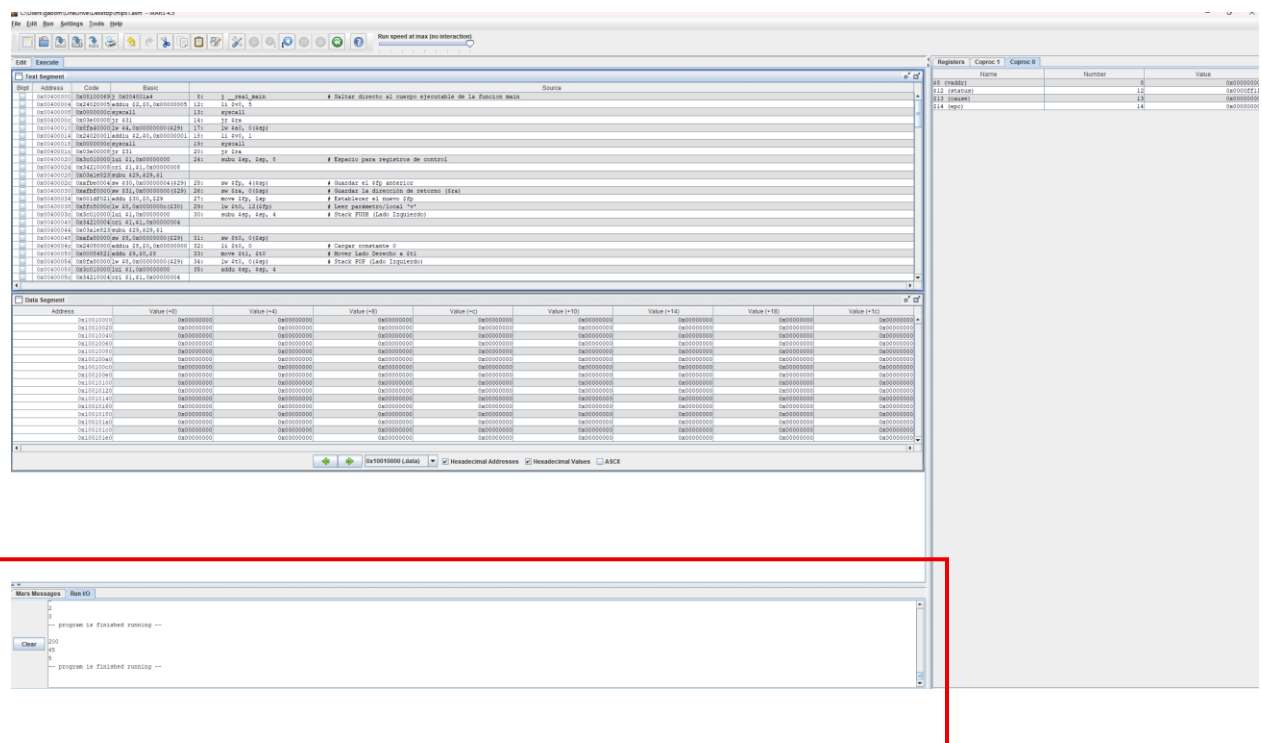
```
file.s
|
| # --- INICIO FUNCIÓN: main ---
|
| real_main:
|   subu $sp, $sp, 8           # Espacio para registros de control
|   sw $fp, 4($sp)            # Guardar el $fp anterior
|   sw $ra, 0($sp)            # Guardar la dirección de retorno ($ra)
|   move $fp, $sp             # Establecer el nuevo $fp
|   subu $sp, $sp, 8           # Espacio para 2 variables locales
|   # -> Asignación a: x
|   jal input                 # Llamar a la función input
|   move $t0, $v0              # Guardar el valor de retorno en $t0
|   sw $t0, -4($fp)           # Guardar en local/parámetro 'x'
|   # -> Asignación a: y
|   jal input                 # Llamar a la función input
|   move $t0, $v0              # Guardar el valor de retorno en $t0
|   sw $t0, -8($fp)           # Guardar en local/parámetro 'y'
|   # -> Pasando argumentos para output
|   # -> Pasando argumentos para gcd
|   lw $t0, -8($fp)           # Pasar puntero heredado del arreglo 'y'
|   subu $sp, $sp, 4           # Argumento PUSH al Stack
|   sw $t0, 0($sp)            # Guardar en local/parámetro 'y'
|   lw $t0, -4($fp)           # Pasar puntero heredado del arreglo 'x'
|   subu $sp, $sp, 4           # Argumento PUSH al Stack
|   sw $t0, 0($sp)            # Guardar en local/parámetro 'x'
|   jal gcd                   # Llamar a la función gcd
|   addu $sp, $sp, 8           # Limpiar argumentos pasados
|   move $t0, $v0              # Guardar el valor de retorno en $t0
|   subu $sp, $sp, 4           # Argumento PUSH al Stack
|   sw $t0, 0($sp)            # Guardar en local/parámetro 'y'
|   jal output                 # Llamar a la función output
|   addu $sp, $sp, 4           # Limpiar argumentos pasados
|   move $t0, $v0              # Guardar el valor de retorno en $t0
|   # --- FINALIZACIÓN LIMPIA DE MAIN PARA MARS ---
|   move $sp, $fp             # Destruir variables locales
|   lw $fp, 4($sp)            # Restaurar $fp anterior
|   addu $sp, $sp, 8           # Liberar espacio de control
|   li $v0, 10                 # Código de servicio 10: Terminar programa
|   syscall                   # Ejecutar salida segura sin retornar a $ra
|
```

Paso 3:

Para correr el código en MARS, se debe presionar el botón de “Compilar” y después en el botón de “ejecutar”, ambos en la parte superior izquierda.



p.D. Para los códigos que requieran un input, se habilitará la escritura en la sección de “RUN I/O” en la parte inferior del programa, donde se deberán introducir los valores de acuerdo con el código insertado.



Sección IV. Apéndice

Para más información de cada sección del compilador, visitar los siguientes documentos en esta misma carpeta:

Lexer

- Proyecto 1 Analizador Léxico Tabla
- Proyecto 1_ Analizador Léxico Reporte

Parser

- EBNF

Analizador Semántico

- Reglas Semánticas de C-

Adicionalmente, los documentos de esta entrega se encuentra en este repositorio:

<https://github.com/Toootiz/Compilador>